

Way of Working

How struct2flow builds software

Luiz Scheidegger · 2026

Why this deck

Two things make this different from "another opinionated framework":

- **The persona team** — named AI personas (Sylvia, Christian, Kathrin, ...) on Claude Code / Codex / Gemini, coordinating via radio-over
- **The blueprint** — a *living operating system*: capabilities derived from production experience, two-way sync to every project

...on top of **eight engineering concerns**, all encoded in tooling:

1. **Architecture** — DDD + Clean + Hexagonal
2. **Lifecycle** — four states, founder-gated
3. **Quality** — Definition of Done, non-negotiable
4. **Observability (MALT)** — Monitoring · Alerting · Logging · Tracing
5. **Security** — secret-scan, SAST, SCA, IaC scan, DAST
6. **Infrastructure as Code** — defined, reviewable, reproducible
7. **Cost** — billable paths capped, logged, alerted, explicit backlog-opt-in
8. **Documentation** — external + internal, in sync with reality, same-commit rule

Everything below is **enforced by tooling**, not memos.

The rules live in code (hooks, scripts, gates) — not in slides.

The thesis

how fast you can find and fix errors.

The difference between good and bad systems
is the **speed-to-fix differential**.

Every section that follows is a way of compressing that gap.

The two **meta-layers** below are what's new; the eight concerns underneath are the engineering substrate they sit on.

The persona team — radio-over

A configurable team of **named personas** (Sylvia · Klaus · Kathrin · ...), each backed by Claude Code / Codex / Gemini / GitHub Copilot. Defined in `AGENT_ROSTER.md`; coordinated via `AGENTS.md` + `AGENT_SIGNAL.md`.

- **One mic at a time** — `Holder = <persona>`, `State` ∈ `IDLE` / `ACTIVE` / `OVER_TO_<NAME>`
- **Persona names prevent same-backing collision** — two Claude Code sessions stay distinguishable (`OVER_TO_SYLVIA` ≠ `OVER_TO_CHRISTIAN`)
- **Read-only + out-of-scope work** allowed in parallel
- **Reactivity:** `Monitor`-based mtime poll, ~2 s latency, zero token cost between events
- **Codex / Gemini dispatched by flipping the signal**, not by direct CLI call
- `HANDOVER.md` lets a fresh prompt resume cold
- **Live feed:** `scripts/agent-activity.sh` — every persona's mic moves *and* tool-by-tool work (orchestrator + subagents + Codex/Gemini) stream in one tail; self-heals across restarts

A configurable team of AI engineers on the same repo — without overwrites or duplicate work.

The blueprint — a living operating system

*Continuously evolving. Derived from production experience.
Intentionally incomplete.*



The evolutionary loop:

1. **Projects create requirements** — real customers, real incidents, real production.
2. **Requirements harden into patterns** — what worked, captured as a recipe.
3. **Patterns become blueprint capabilities** — promoted upstream via `blueprint a2bp`.
4. **Capabilities become defaults** — for the next project *and* for every existing one, pulled via `blueprint pull`.

The flow is bidirectional. Existing projects ship patterns *up*; every project — current *and* future — pulls capabilities back *down*. A lesson learned in storm2flow improves linkedin-watcher the same week.

The blueprint contains **only what has been proven in production** — that's why what it does carry, you can rely on.

The blueprint — one repo, every project

Every struct2flow project is **forked from a single blueprint**:
all eight concerns below, plus the agent infra, live in one git repo.

- **Bootstrap** — `new-project.sh acme` copies the blueprint into a new project directory, substitutes placeholders, and records the source SHA in `.blueprint-source`.
- **Pull** — `blueprint drift` shows what's changed in the blueprint since the project's last sync. `blueprint pull` brings the improvements forward.
- **Push** — `blueprint a2bp <file>` apply-to-blueprint: when a generic improvement lands in a project, it travels back to the blueprint so *every other project* inherits it next time they pull.

A rule tightened once in any project benefits every project. The blueprint is the multiplier.

Blueprint sync — the CLI

A single `blueprint` command, four subcommands:

```
blueprint drift          # what's drifted vs blueprint HEAD + commits since bootstrap
blueprint pull [FILE...] # pull blueprint changes forward (interactive, founder approves)
blueprint a2bp FILE [...] # apply-to-blueprint: stage a generic improvement upstream
blueprint files          # list the blueprint-managed files (single source of truth)
```

What's managed — `CLAUDE.md`, `STACK_DEFAULTS.md`, `Brewfile`, every recipe doc (`OBSERVABILITY.md` / `SECURITY.md` / `INFRASTRUCTURE.md`), `DoD.md`, the agent scripts, the pre-push hook, this deck itself.

What's NOT managed — `project_config_*.md` (templates seeded once at bootstrap, then drift on purpose), `BUGS.md`, `HANDOVER.md`, `AGENT_SIGNAL.md`, all source code.

The agent calls `blueprint drift` on every wake. Drift between blueprint and project is treated like drift between code and prod: **detected, not assumed away**.

1 • Architecture

Default for every struct2flow project:

- **Domain-Driven Design + Clean Code + Clean Architecture**
- **Hexagonal (ports & adapters)** whenever non-trivial external integration exists
- **Layering:** `domain → application → ports → adapters` — dependencies point inward only
- Override allowed for thin CRUD or one-shot scripts — state why in one sentence

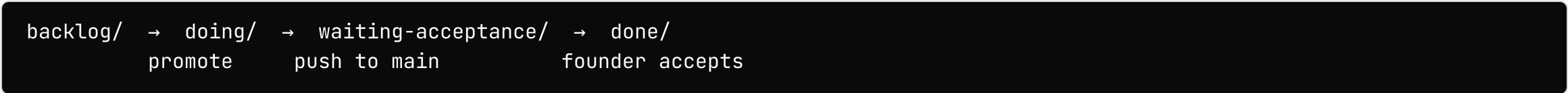
Source of truth: `STACK_DEFAULTS.md` § Architecture

1 • Architecture — why this default

- **Domain stays pure and testable** — no framework imports in the core
- **Adapter swaps are bounded** — DynamoDB → Mongo, REST → GraphQL, sync → event-driven
- **Infrastructure pushed to the edges** — core survives stack changes
- **Coverage is measured whole-tree, tiered by layer** — domain/app ≥90%, adapters ≥80%, CLI ≥75% (a high % over a curated slice is theatre)

The point isn't dogma. It's that the *most expensive layer to get wrong* (the domain) is the *easiest to keep clean* — if you keep frameworks out of it.

2 • Lifecycle — four founder-gated states



State	What lives here
backlog/	Parked work — every row marked KEEP / DEFER / OBSOLETE
doing/	Active. Bugs, plans, in-flight work
waiting-acceptance/	Pushed to main, awaiting founder test
done/	Founder-accepted. Source of truth for <i>delivered</i>

2 • Lifecycle — the rules that matter

- **backlog/ is not a graveyard** — every parked row carries an explicit re-open trigger or **OBSOLETE** marker
- **waiting-acceptance/ is the only path into done/** — never skip
- **done/ is founder-only** — agents never auto-promote
- **Reopen path:** rejected acceptance → row goes back to **doing/**
- **Grooming pass:** founder-led session pulls parked items into **doing/**

*The lifecycle answers "what has been delivered?" —
not "what has been merged?" Those are different questions.*

3 • Quality — Definition of Done

`docs/DoD.md` is the canonical handoff gate. Eight checklist sections.

Non-optional rules:

- **Two-commit pattern** — reproducer test (failing) → fix
- **Coverage thresholds** — whole-tree, tiered: domain/app $\geq 90\%$, adapters $\geq 80\%$, CLI $\geq 75\%$ (brownfield ratcheted)
- **ESLint + Prettier** — both blocking, independent gates
- **Pre-push ≤ 30 s** wall-clock — slower tests go to CI
- **Snapshots are approval-based** — CI never runs with `-U`

3 • Quality — the two-commit pattern

Every product/runtime bug fix lands as **two** commits, in this order:

1. `test(BUG-XXX): minimal reproducer (failing)`
2. `fix(BUG-XXX): <fix>`

The reproducer **must fail** on the parent commit.

Verified by stash-test-restore — `git log` proves it.

"I added a regression test" is only credible when `git log` shows the test failing before the fix.

This single rule kills the "I'll write the test later" anti-pattern at source.

3 • Quality — the pre-push gate

The shared `.github/hooks/pre-push` hook **blocks the push** if any step fails:

1. Build (e.g. `tsc` — catches missing imports)
2. Lint (`--max-warnings` ratcheted; never loosen)
3. **Prettier** `--check` — fails on any unformatted file
4. Tests + **coverage gate** (project's threshold)
5. Project-specific guards (placeholder, asset, release-notes...)

Same hook, every project. No "I'll skip pre-push just this once."

4 • Observability — MALT

Monitoring · **A**lerting · **L**ogging · **T**racing —
a first-class concern, not an afterthought.

Since quality is non-negotiable, the quality of the working software is fundamental. The difference between good and bad systems is how quickly we can find and fix errors — the speed-to-fix differential.

Four non-negotiable capabilities per project →

4 • Observability — the four capabilities

1. **Every error path is captured.**

No silent swallowing. No `try/catch` that returns success. If it broke, it logs.

2. **Every captured error is agent-queryable** without human ferrying.

Documented retrieval path; agent uses it *before* asking the founder.

3. **Every shipped capability is alertable** in production.

Threshold + destination declared in `project_config_dod.md`.

4. **The agent diagnoses first.**

Humans pinged only when the agent is genuinely stuck.

4 • Observability — recipes per stack

`docs/OBSERVABILITY.md` — concrete patterns per runtime:

- **Recipe A — AWS / serverless** (CloudWatch JSON logs, EMF metrics, X-Ray tracing)
- **Recipe B — desktop / CLI** (structured local logs, `--diagnose` flag)
- **Recipe C — third-party hosted** (Sentry / Datadog wiring)

Each recipe delivers the same four capabilities — they just differ in plumbing.

Worked example in production: storm2flow's MALT pattern (`FEATURE-006`).

4 • Observability — wired into the DoD

For every new user-facing route, command, or job — `DoD.md` §6.1 requires:

- [] **Error capture** — structured boundaries; no silent swallowing
- [] **Agent-readable retrieval path** — one command gets the data
- [] **Alert** — threshold + destination per `project_config_dod.md`
- [] **Runbook link** if the alert can wake someone up

A feature without observability is not a feature done.

5 • Security — four non-negotiable capabilities

Quality of working software degrades to zero the moment something is exploited in production.

Same principle as MALT, applied at a different timescale:

1. **No secrets in code or git history.**

`gitLeaks` blocks the push; leaked secrets get rotated, not unstaged.

2. **Static analysis catches OWASP top-10** at commit time.

Semgrep + lint security plugins, pre-push, with justified suppressions only.

3. **Deps and infra are scanned continuously.**

`osv-scanner` per push; `trivy config` before deploy; nightly CVE watcher.

4. **The agent fixes findings autonomously.** Same diagnosis-first pattern as MALT.

5 • Security — recipes per stack

`docs/SECURITY.md` — same recipe-per-stack model as `OBSERVABILITY.md` :

- **Recipe A — AWS / serverless**
gitleaks + Semgrep + osv-scanner in pre-push;
trivy config + ZAP baseline in CI; nightly active scan + new-CVE watcher.
- **Recipe B — Local app / desktop / CLI**
Same SAST + SCA + secret-scan. Signed releases instead of DAST
(no remote surface to scan).
- **Recipe C — Containerized service**
Above plus `trivy image` (blocks vulnerable base images) and ZAP
against the service's HTTP surface.

Threat model + thresholds per project: `project_config_security.md` .

6 • Infrastructure as Code — four non-negotiable capabilities

Quality of working software depends on the environment matching its definition.

Drift between code and prod is the silent killer of reproducibility.

1. **Everything in prod is in code.**

No console clicks; no "I'll codify it later". Imported or deleted within the week.

2. **Every change is reviewable as a diff.**

`cdk diff` / `terraform plan` / `helm diff` is the review artifact, not the source.

3. **Environments are reproducible from the same code.**

New env = one row of config + one command. Never a hand-crafted sandbox.

4. **Drift is detected, not assumed away.**

Nightly diff; out-of-band changes are codified or reverted, never ignored.

6 • IaC — recipes per stack

`docs/INFRASTRUCTURE.md` — same recipe-per-stack model as `SECURITY.md` / `OBSERVABILITY.md`:

- **Recipe A — AWS-first (CDK TypeScript)** — struct2flow default
CodePipeline-driven applies; manual approval before prod;
CDK Nag + Infracost gating posture + cost.
- **Recipe B — Multi-cloud / portable (Terraform / Pulumi)**
S3 + DynamoDB state/lock; pipeline-only prod-apply;
`tfsec` + `infracost` gating.
- **Recipe C — Kubernetes-native (Helm + ArgoCD / Flux)**
GitOps; ArgoCD pulls cluster state from a branch;
`OutOfSync` is the native drift detector.

Environments / ownership / cost ceilings / rollback: `project_config_infra.md`.

7 • Cost — four non-negotiable capabilities

Working software that quietly bankrupts the founder is broken software.

Any billable code path (LLM, search, OCR, storage, egress) burns real money. The failure mode is **silent** until the bill arrives.

1. Every billable path declares a budget cap.

Per-call / per-tick / per-day, in code. Cap reached → **halt the loop**, not "log and continue".

2. Every invocation logs structured spend.

`{model, input_tokens, output_tokens, usd}` — so the agent can answer "how much did we spend today?" without a vendor dashboard.

3. Alerts fire on the rising edge.

Same Slack lane as MALT. Once when cap is hit — not every tick after.

4. Backlog-replay needs explicit opt-in.

`--catch-up` / `--first-run` / `--replay-since=...` — never implicit. "Process everything piled up" is opt-in by the operator who priced it.

7 • Cost — recipes per stack

Mechanism is project-specific; the **capabilities** are not. Declared in `project_config_overview.md` §"Cost stack".

- **LLM-backed agent** — token-cost SDK helper; per-tick budget gate halts further calls when cumulative > cap; structured `{model, tokens, usd}` log; transition-edge Slack alert when daily cap hit. Freshness gate + dedup store are the **upstream** defences; the cap is the last-line backstop.
- **External API consumer** (Twilio, Stripe webhook fan-out) — same shape: declared cap, per-call cost logged, alert on transition, explicit opt-in for backlog replay.
- **Storage / egress** — per-tick byte budget; structured per-call size log; alert on transition; explicit opt-in for large historical syncs.

***Real incident:** linkedin-watcher-agent took a single **\$10 hit** when a fetcher bug fix (`BUG-001`) unblocked a 374-post backlog before the freshness gate existed (`BUG-003`). The fix was correct; the missing guardrail made it expensive. Now: design the cap + the explicit-opt-in flag together, not retrofit after the first surprise bill.*

8 • Documentation — four non-negotiable capabilities

Working software with stale docs is software no one trusts.

The **silent** failure mode: no exception, no alert, just compounding embarrassment until a customer notices.

1. Same-commit rule for external docs.

User-facing change → README, release notes, feature page, help article, pricing — all touched in the **same commit**, not "same PR".

2. Internal artefacts move with the state they describe.

`FEATURES.md`, `findings.md`, threat-model entries, ADRs, runbooks — updated *as* the code changes, not after.

3. Blueprint-level changes update the deck + recipe doc in the same commit.

`docs/way-of-working.md` is the pitch surface. It has self-violated **four times this week** (Cost; "six" → "seven"; Documentation itself; persona-team framing). Fix in flight: `docs/A2BP_PLAYBOOK.md`.

4. Drift is detected, not assumed away.

Promotion criteria for the sync list; pre-push grep-based hints for known drift patterns; handoff-time checklist box that refuses the mic flip if any sync-list file is stale.

8 • Documentation — recipes per project shape

`docs/DOCUMENTATION.md` — same recipe-per-stack model as `OBSERVABILITY.md` / `SECURITY.md` :

- **Recipe A — Single-repo README-only**

Small projects, CLIs, internal tools. Sync list: README + release-notes + FEATURES + findings. Append-only release notes; same-commit rule on every push.

- **Recipe B — Static-site marketing + docs**

Customer-facing app + `docs-site/` (Mintlify / Astro Starlight / Nextra). Per-page-type sync rules; OpenAPI generated from code; PR preview deploys.

- **Recipe C — Customer help portal + status + privacy/TOS**

Mature SaaS. Help portal (Intercom / Zendesk) with `help-index.md` in repo; public status page; **versioned** legal docs; ADRs; runbooks co-located with alerts.

Two-table sync list (External / Internal) in `project_config_dod.md` §"Doc-sync list". DoD §6.4 is the per-push gate.

What this gets you

If you are...	What you get
A customer	Software that ships fast <i>and</i> is observable when it breaks. Speed-to-fix is the differential.
An investor / partner	An operational moat — repeatable, auditable, agent-native dev practice.
A collaborator	A clear bar at every gate. No tribal knowledge. The hooks tell you the rules.
The founder	The same discipline applied to every project, with the boring parts automated.

The compounding effect

Each rule is small. The compounding is the point:

- **Hexagonal** keeps the domain pure → tests stay cheap.
- **Cheap tests** make the **90% coverage bar** affordable.
- **High coverage** + **two-commit pattern** means regressions are caught at commit, not in prod.
- **MALT** means the ones that *do* reach prod are diagnosed by the agent in minutes.
- **Lifecycle gates** mean "done" actually means delivered, not merged.

The output is a system where **speed-to-fix shrinks every week**.

...and because every project is forked from the same blueprint and sync'd both ways, **every project gets every improvement** — within the same week one project learned it.

Where to read more

- `README.md` — what's in the blueprint, how to spawn a project
- `CLAUDE.md` — long-form agent protocol
- `docs/DoD.md` — Definition of Done (the handoff gate)
- `STACK_DEFAULTS.md` — architecture + stack defaults
- `docs/OBSERVABILITY.md` — MALT recipes per runtime
- `docs/SECURITY.md` — security recipes per runtime
- `docs/INFRASTRUCTURE.md` — IaC recipes per runtime
- `docs/DOCUMENTATION.md` — doc-sync recipes (internal + external)
- `scripts/blueprint` — sync CLI: `drift` / `pull` / `a2bp` / `files`
- `project_config_*.md` — per-project overrides

All open source at:

github.com/LuizStruct2Flow/blueprint

Thank you

Luiz Scheidegger

luiz@struct2flow.com

Questions?